

Trabalho Prático 1

Introdução à Inteligência Artificial (DCC 642/UFMG)

Daniel Vitor Rabelo Rodrigues, Manuela Monteiro Fernandes de Oliveira

daniel.rodrigues@dcc.ufmg.br, manuela.monteiro@dcc.ufmg.br

1. Introdução e Objetivo

O Trabalho Prático 1 (TP) teve por finalidade a implementação de um agente autônomo de Inteligência Artificial (IA) para jogar partidas adversariais de Lige-4/Connect4, através do algoritmo Minimax com profundidade limitada e suas variações com poda Alfa-Beta e Iterative Deepening. Com isso, o TP objetivou-se em nos ensinar a implementação de tais algoritmos e como as mudanças feitas no seu desenvolvimento podem influenciar a sua performance ao longo das jogadas.

2. Metodologia

Para realização do Trabalho Prático 1, nós implementamos o agente em etapas, indo do algoritmo mais simples (versão tradicional do Minimax) até o mais complexo (Minimax com poda Alfa-Beta e Iterative Deepening).

Nesse sentido, inicialmente implementamos o algoritmo tradicional com profundidade limitada (função *minimax*), e utilizamos apenas a valoração no centro do tabuleiro – que considerava as diferentes formas de ganhar o jogo dada aquela posição – como heurística para avaliar os casos não terminais em que a profundidade máxima era atingida (função *evaluate*). Porém, percebemos que o algoritmo não estava sendo muito eficiente, pois sempre escolhia a primeira coluna da esquerda para fazer as suas jogadas, mesmo quando essa não era a jogada mais estratégica. Diante disso, decidimos mudar a função de avaliação heurística para que ela considerasse também a quantidade de sequências de duplas/triplas/quádruplas abertas (na horizontal, vertical e diagonal), tanto do jogador do turno atual como do oponente, e avaliasse cada caso de forma diferente, para que o algoritmo fosse capaz de escolher os estados que mais otimizassem o seu objetivo. Além disso, inicialmente a heurística possuía comandos de `print()`, que foram colocados para nos auxiliar a entender os cenários traçados antes de cada jogada, mas que decidimos retirar, pois estavam fazendo com que o algoritmo se tornasse alguns milissegundos mais lento e, com isso, atingisse o tempo máximo permitido por jogada (parâmetro *max_time*) antes de avaliar todos os possíveis estados da árvore. Após essas mudanças, o agente passou a executar as jogadas de maneira mais efi-

ciente e estratégica e, então, pudemos implementar as etapas de poda Alfa-Beta e Iterative Deepening em sequência. Para a implementação da poda Alfa-Beta e Iterative Deepening, foram adicionados dois novos agentes ao arquivo `server.py`, para habilitar partidas entre os três modelos. O algoritmo Alfa-Beta foi implementado como uma otimização direta do Minimax tradicional, preservando a mesma lógica mas introduzindo poda de ramos incapazes de influenciar a decisão final. Para isso, foi criada a função *minimax_prune()*, que passou a contar com parâmetros *alpha* (melhor valor garantido para o jogador MAX) e *beta* (melhor valor garantido para o MIN), permitindo interromper a expansão de estados quando uma jogada já se mostrava inferior a alternativas previamente encontradas.

O Iterative Deepening foi desenvolvido como uma extensão do Alfa-Beta, realizando múltiplas buscas sucessivas com profundidade crescente até que o limite de tempo configurado fosse atingido. A implementação reutiliza valores de iterações anteriores em caso de timeout, mas seguiu a decisão de projeto de sempre confiar na iteração mais recente, quando concluída, substituindo o valor ótimo encontrado independente de uma aparente vantagem do valor da iteração anterior. Como o algoritmo executa várias buscas Alfa-Beta consecutivas, sua integração foi feita de forma modular, reutilizando integralmente o mecanismo de poda já implementado.

3. Experimentos e Resultados

Em todos os testes que não haviam um limite de tempo especificado, foi fixado um limite de 3000ms, o mesmo que será utilizado na competição entre os agentes de diferentes duplas. Em geral, quando em paridade de capacidade entre os agentes, foi observado um viés de favorecimento ao agente que começa a partida, então todos os testes de confrontos foram realizados com dez partidas, dentre as quais cada agente jogava cinco realizando a primeira jogada.

3.1. Minimax vs. Aleatório

Como esperado, o algoritmo Minimax obteve cem por cento das vitórias contra um algoritmo sem racionalidade. Foi possível observar claramente o crescimento exponencial de complexidade na quantidade de nós visitados ao longo da progressão de 2 até 5 de profundidade na busca.

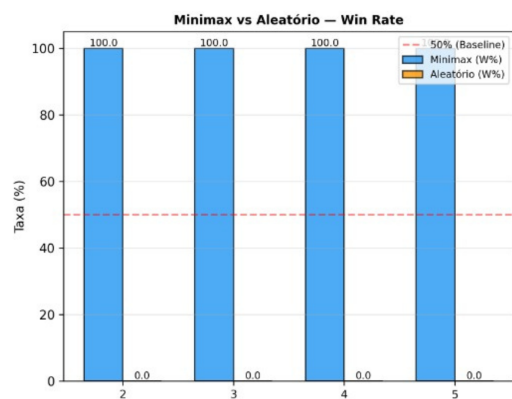


Figure 1: Taxa de Vitória: Minimax vs. Aleatório.

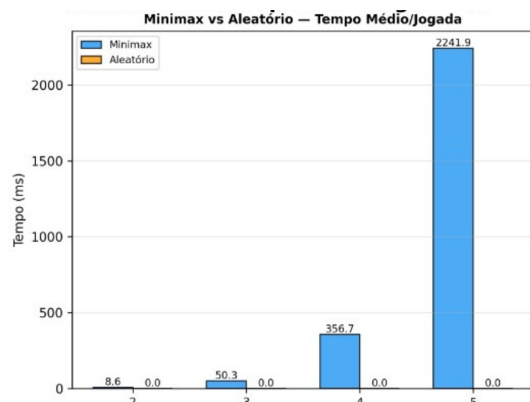


Figure 2: Tempo Médio: Minimax vs. Aleatório.

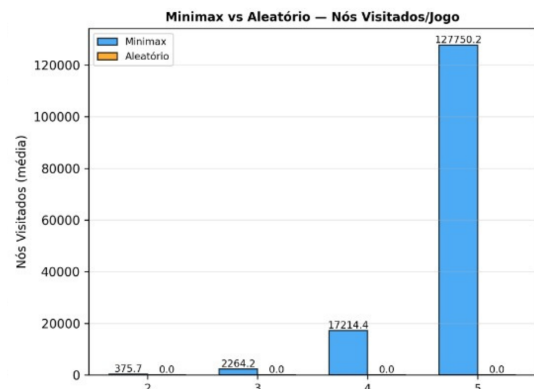


Figure 3: Nós Visitados: Minimax vs. Aleatório.

3.2. Minimax vs. Poda Alfa-Beta

Em geral os dois algoritmos apresentaram um desempenho parecido, obtendo cerca de metade das vitórias na maioria das profundidades, e obtendo uma anômala porém análoga situação de equilíbrio na profundidade 3, onde houveram dez empates em dez partidas. Entretanto, pode ser observada

como a quantidade de nós visitados e tempo médio das jogadas, que chegou inclusive a ser maior para a poda alfa-beta na profundidade 2, cresceu de maneira muito mais significativa no minimax comum, sendo mais de três vezes maior em profundidade cinco.

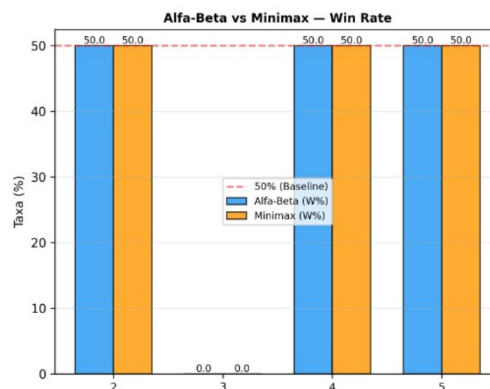


Figure 4: Taxa de Vitória: Minimax vs. Poda Alfa-Beta.

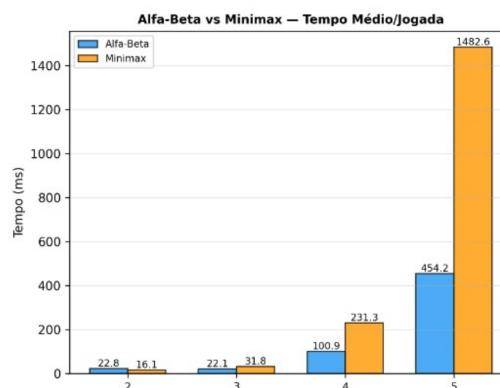


Figure 5: Tempo Médio: Minimax vs. Poda Alfa-Beta.

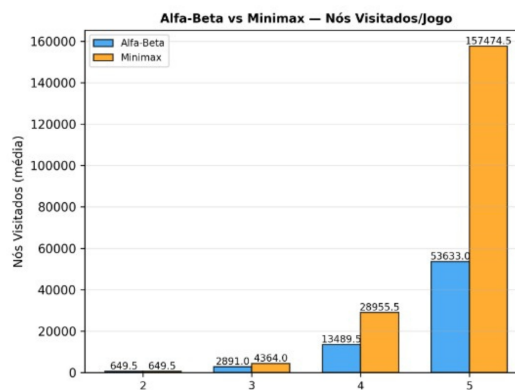


Figure 6: Nós Visitados: Minimax vs. Poda Alfa-Beta.

3.3. Poda Alfa-Beta vs. Iterative Deepening

Os dois algoritmos se mostraram similares em quantidade de nós visitados e tempo médio de jogadas, uma vez que ambos são baseados na poda alfa-beta. Já nos resultados do confronto, com 1000ms de limite de tempo, o algoritmo de poda alfa-beta puro foi invicto, por aproveitar melhor o tempo limitado ao lidar com apenas uma iteração. Entretanto, com 2000ms de limite, os dois agentes se equipararam em uma taxa de vitória de metade das partidas.

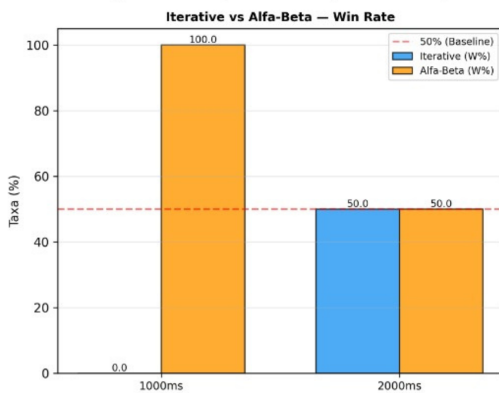


Figure 7: Taxa de Vitória: Poda Alfa-Beta vs Iterative Deepening.

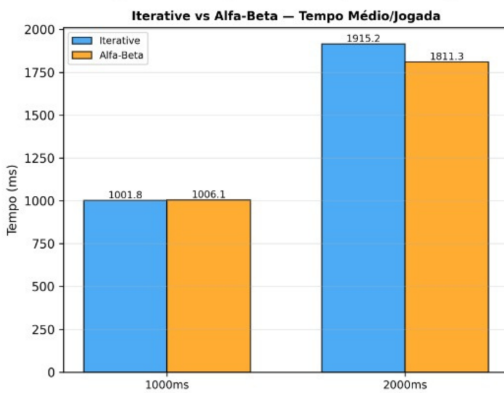


Figure 8: Tempo Médio: Poda Alfa-Beta vs Iterative Deepening.

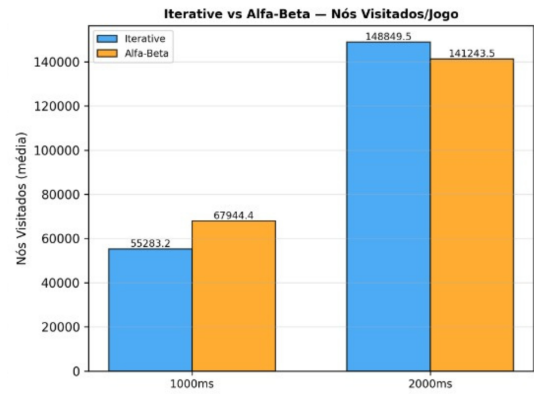


Figure 9: Nós Visitados: Poda Alfa-Beta vs Iterative Deepening.

3.4. Jogando contra os Agentes

Após algumas partidas contra os agentes, a principal impressão que tive é que seus primeiros passos parecem previsíveis e lentos, uma vez que a grande quantidade de movimentos possíveis exige bastante processamento, e a limitação de profundidade gera decisões menos surpreendentes, porém ao decorrer das partidas, os agentes tendem a ficar mais rápidos a cada jogada, e a tomar decisões cada vez mais precisas, ao ponto de ser difícil, ou até impossível, bater a máquina com tempo e profundidade suficientemente ajustados.

4. Discussão

A partir do desenvolvimento do TP e dos experimentos realizados, foi possível perceber que o algoritmo Minimax tradicional com limite de tempo – com as heurísticas de valorização do centro do tabuleiro, contagem de sequências (duplas, triplas e quádruplas abertas) e penalidades para oportunidades do oponente – é capaz de verdadeiramente jogar de maneira estratégica contra um oponente humano e, em muitos casos, vencê-lo. Porém, essa versão do algoritmo demanda mais tempo para ser executada e carece de otimizações, visto que na maioria das vezes ele gasta todo o tempo limite disponível para realizar uma manobra, percorrendo inteiramente a árvore de estados em busca de soluções potencialmente melhores. Diante disso, a poda Alfa-Beta se mostrou muito interessante, pois permite que o algoritmo ignore os nós das sub-árvores que não influenciam na decisão final, sem perder a jogada ótima, e, com isso, consiga retornar qual é a melhor jogada com mais agilidade e gastando menos recursos computacionais. A estratégia de Iterative Deepening também se mostrou interessante, por empatar com o algoritmo da poda Alfa-Beta e, ao mesmo tempo, possuir a vantagem de sempre retornar a melhor jogada para o agente, mesmo que o limite de tempo chegue ao fim, por explorar os níveis da árvore iterativamente e guardar a jogada ótima.

5. Conclusão

O desenvolvimento deste Trabalho Prático permitiu compreender, na prática, o impacto de diferentes técnicas de otimização em algoritmos de busca adversarial para o jogo Lige-4.

Primeiramente, ficou evidente que o sucesso do agente depende não apenas do algoritmo de busca, mas fortemente da qualidade da função de avaliação. A transição de uma heurística simples (baseada apenas no centro do tabuleiro) para uma avaliação mais complexa, que contabiliza sequências de duplas, triplas e quádruplas e penaliza as oportunidades do oponente, foi o que o tornou verdadeiramente competitivo e estratégico, mesmo com uma arquitetura mais simples, em comparação com as implementações otimizadas.

Em termos de algoritmo, a combinação da poda Alfa-Beta com o Iterative Deepening provou ser a arquitetura ideal para este cenário. A poda Alfa-Beta demonstrou ser essencial para poupar recursos computacionais e tempo, cortando sub-árvores irrelevantes sem perder a garantia da jogada ótima. Paralelamente, o Iterative Deepening complementou essa eficiência, garantindo que a jogada ótima seja escolhida mesmo em cenários em que o tempo é escasso, como foi o caso da restrição rígida de 3000ms dos testes. Além disso, a simples remoção de comandos de `print()` também se mostrou uma lição valiosa sobre como operações de I/O (entrada e saída) podem ser um gargalo de desempenho em algoritmos com limite de tempo.

Apesar de o agente atual ser capaz de enfrentar oponentes humanos com alta taxa de sucesso, uma melhoria que poderia ser implementada é a de ordenar as colunas do tabuleiro com base em sua relevância para uma boa jogada, antes de passá-las para o algoritmo de Alfa-Beta Pruning, pois este é mais eficiente quando os nós mais valiosos são avaliados primeiro. Nesse sentido, priorizar a expansão de nós que jogam nas colunas centrais ou que bloqueiam/formam ameaças imediatas aumentaria drasticamente o número de cortes na árvore e, assim, garantiria um algoritmo mais eficiente ao possibilitar que a decisão do agente fosse tomada de forma mais rápida e eficaz.

References

DataCamp Team. 2025. Implementing the Minimax Algorithm for AI in Python. <https://www.datacamp.com/pt/tutorial/minimax-algorithm-for-ai-in-python>. [Online; acessado em 10-maio-2026].

DeNero, J., and Klein, D. 2026. CS188 Textbook - Games: Minimax. <https://inst.eecs.berkeley.edu/~cs188/textbook/games/minimax.html>. [Online; acessado em 03-maio-2026].

Science Buddies. 2024a. Simple Explanation of the Minimax Algorithm with Alpha-Beta Pruning with Connect 4. YouTube: <https://youtu.be/DV5d31z1xTI>. [Online; acessado em 10-maio-2026].

Science Buddies. 2024b. Connect 4 AI Player using Minimax Algorithm with Alpha-Beta Pruning: Python Coding Tutorial. YouTube: <https://youtu.be/rbmklqtVEmg>. [Online; acessado em 10-maio-2026].

Spanning Tree. 2023. Minimax: How Computers Play Games. YouTube: <https://youtu.be/SLgZhpDsrfc>. [Online; acessado em 10-maio-2026].

Wikipedia contributors. 2026. Minimax — Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/wiki/Minimax>. [Online; acessado em 03-maio-2026].